

Lab 1 Extra: 格式化输入 (scanf) 的实现

准备工作：创建并切换到 lab1-extra 分支

请在自动初始化分支后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab1-extra
```

初始化的 lab1-extra 分支基于课下完成的 lab1 分支，并且在 tests 目录下添加了 lab1_scan 样例测试目录。

题目背景

在操作系统内核中实现类似于 scanf 的格式化输入功能时，我们需要处理来自外部设备（如键盘、串口）的字符流。由于我们缺乏庞大的标准 C 库，你需要亲手实现一个精简版的格式化解析函数。

本题需要精准实现标准格式控制符（%c、%u、%s）对字符流的“吞吐行为”。由于在解析变长数据（如读取数字和字符串）时，程序通常需要多读一个不属于该格式的字符才知道当前读取已经结束。因此，如何妥善保管这个“多读出来”的字符，不让它被丢弃，使得下一个格式符能继续正常解析，是本次实现的关键。

核心解析规则（务必仔细阅读）

你需要实现以下三种格式说明符。请严格遵循它们对“前导空白符”和“结束边界”的处理逻辑：

空白符定义：空格 ' '、水平制表符 \t、换行符 \n、回车符 \r。

1. %c（单字符）

- **前导空白：**绝不跳过。它会严格读取当前的下一个字符，无论是不是空白符。
- **结束边界：**读取 1 个字符后立即结束。
- **状态要求：**由于字符已被 %c 实体消耗，你需要强制要求系统在下一次解析时重新去读取新字符（即将缓冲区置为无效）。

2. %u（无符号十进制整数）

- **前导空白：**必须跳过所有的前导空白符。
- **解析行为：**匹配连续的数字（支持跳过前导加号 +，如 +10 解析为 10）。如果遇到的第一个非空白字符就不是数字或 +，直接赋值为 0 并结束。
- **结束边界：**遇到第一个非数字字符时停止。
- **状态要求：**导致停止的那个“非数字字符”必须原封不动地留在缓冲区中，等待下一个格式符处理。
- **数值范围：**[0, 2147483647) (小于 int 范围)

3. %s（标准字符串）

- **前导空白：**必须跳过所有的前导空白符。
- **解析行为：**连续读取非空白字符，并将其存入指针指向的内存中。
- **结束边界：**遇到第一个空白符或 \0 时停止，并在末尾自动补充 \0 封口。

- **状态要求**：导致停止的那个“空白符”必须原封不动地留在缓冲区中，等待下一个格式符处理。
- 保证 0<字符串长度<20

你的任务 (Task List)

请按顺序执行以下修改任务：

任务 1：添加头文件声明

在 `include/printk.h` 中加入以下声明（要加到 `#define _printk_h_` 和 `#endif /* _printk_h_ */` 的内部）：

```
// include/printk.h
int scan(const char *fmt, ...);
```

在 `include/print.h` 中加入以下声明（要加到 `#define _print_h_` 和 `#endif` 的内部）：

```
// include/print.h
typedef void (*scan_callback_t)(void *data, char *buf, size_t len);
int vscanfmt(scan_callback_t in, void *data, const char *fmt, va_list ap);
```

任务 2：实现顶层包装函数

在 `kern/printk.c` 中添加以下代码，实现字符流获取与 `scan` 的包装：

```
// kern/printk.c
// 帮助函数，用于从控制台获取长度为 len 的字符，并写入 buf 中。
void inputk(void *data, char *buf, size_t len) {
    for (int i = 0; i < len; i++) {
        // 1. 轮询等待键盘输入，没有输入就死等
        while ((buf[i] = scancharc()) == '\0') {
        }

        // 2. 统一回车符：将 '\r' 转成 '\n'，方便后续的 vscanfmt 识别空白符
        if (buf[i] == '\r') {
            buf[i] = '\n';
        }

        // 3. 终端回显：读到了什么，就立刻印在屏幕上
        printcharc(buf[i]);
    }
}

int scan(const char *fmt, ...) {
    /* ----- */
    /* Lab 1-extra: Your code here. (1/4) */
    /* 提示：
    1. 仿照 printk 的实现，使用 va_list 处理可变参数。
    2. 调用 vscanfmt（传入刚写好的 inputk 回调函数）。
    3. 本函数的返回值 ret 应为 vscanfmt 的执行结果。
    */
}
```

```

*/
/* ----- */

}

```

任务 3：完成核心解析状态机

在 `lib/print.c` 中添加并补全以下代码。

注：骨架代码已经提供了单字节缓冲区 `ch` 和有效标志位 `ch_valid` 的定义，请利用它们完成【核心解析规则】中要求的行为逻辑。

```

// lib/print.c
#include <stdarg.h>

// =====
// 辅助函数：确保缓冲区里有字符
// =====
static inline void ensure_char(scan_callback_t in, void *data, char *ch, int
*ch_valid) {
    if (!*ch_valid) {
        in(data, ch, 1);
        *ch_valid = 1;
    }
}

// =====
// 辅助函数：跳过前导空白符
// =====
static inline void skip_whitespace(scan_callback_t in, void *data, char *ch, int
*ch_valid) {
    ensure_char(in, data, ch, ch_valid);
    while (*ch == ' ' || *ch == '\t' || *ch == '\n' || *ch == '\r') {
        // 直接覆盖当前字符，不需要改 ch_valid 状态，因为它一直握着有效的最新字符
        in(data, ch, 1);
    }
}

// =====
// 处理 %c：读第一个输入字符，无论是不是空白
// =====
void scan_c(scan_callback_t in, void *data, char *ch, int *ch_valid, char *cp) {
    /* ----- */
    /* Lab 1-extra: Your code here. (2/4) */
    /* 提示：
    1. 使用辅助函数确保缓冲区里有字符（注意：%c 绝不能跳过空白符！）
    2. 将读取到的字符存入 cp 指向的内存。
    3. 核心：因为该字符已被 %c 实体吃掉，必须将 缓存标志位 置为无效(值为0)。
    */
    /* ----- */

}

// =====
// 处理 %s：跳过前导空白，读到空白符终止

```

```
// =====
void scan_s(scan_callback_t in, void *data, char *ch, int *ch_valid, char *cp) {
    /* ----- */
    /* Lab 1-extra: Your code here. (3/4) */
    /* 提示:
    1. 使用辅助函数跳过所有的前导空白符。
    2. 连续读取非空白字符并存入 cp(可以在循环内部使用 *cp++ = *ch 进行赋值)。
    3. 遇到空白符或 '\0' 时停止读取。
    4. 别忘了在字符串末尾添加 '\0' 封口。
    5. 停下时: ch 里必须正好握着导致停止的“空白符”，且保持 缓存标志位 置为有效(值为1)。
    */
    /* ----- */

}

// =====
// 处理 %u: 跳过前导空白, 读到非数字终止
// =====
void scan_u(scan_callback_t in, void *data, char *ch, int *ch_valid, int *ip) {
    /* ----- */
    /* Lab 1-extra: Your code here. (4/4) */
    /* 提示:
    1. 定义变量用于累加数字计算结果。
    2. 使用辅助函数跳过所有的前导空白符。
    3. 兼容可选的前导 '+' 号(至多1个)(如果有, 调用 in() 越过它)。
    4. 如果遇到的第一个非空白字符就不是数字或 '+', 直接赋值为 `0` 并结束。
    5. 连续读取数字字符('0'-'9'), 并 计算 十进制数值。
    6. 遇到非数字字符时停止读取, 并将结果存入 ip。
    7. 停下时: ch 里必须正好握着导致停止的“非数字字符”，且保持 缓存标志位 置为有效(值为1)。
    */
    /* ----- */

}

// =====
// 主入口: vscanfmt
// =====
int vscanfmt(scan_callback_t in, void *data, const char *fmt, va_list ap) {
    char ch;
    int ch_valid = 0; // 缓存标志位
    int ret = 0;

    while (*fmt) {
        if (*fmt == '%') {
            fmt++; // 跳过 '%'

            switch (*fmt) {
                case 's':
                    scan_s(in, data, &ch, &ch_valid, va_arg(ap, char *));
                    ret++;
                    break;

                case 'u':
                    scan_u(in, data, &ch, &ch_valid, va_arg(ap, int *));
                    ret++;
                    break;
            }
        }
        fmt++;
    }

    return ret;
}
```

```

case 'c':
    scan_c(in, data, &ch, &ch_valid, va_arg(ap, char *));
    ret++;
    break;

}
}
fmt++;
}
return ret;
}

```

样例输出 & 本地测试

对于下列样例：

```

void mips_init(u_int argc, char **argv, char **penv, u_int ram_low_size) {
    unsigned int u_basic;
    char c_basic;
    char s_basic[30];

    printk("== Basic Tests ==\n");

    // 基础测试 1：单独测试 %u （测试前导空格过滤和 '+' 号识别）
    // 模拟键盘输入： " +8906x" （注意：遇到非数字 'x' 时停止解析）
    scan_k("%u", &u_basic);
    printk("Basic %u: %d\n", u_basic);

    // 基础测试 2：单独测试 %c
    // 模拟键盘输入： "k"
    scan_k("%c", &c_basic);
    printk("Basic %c: '%c'\n", c_basic);

    // 基础测试 3：单独测试 %s （测试前导空白过滤和遇到空白符停止）
    // 模拟键盘输入： " MOS_kerne l "
    scan_k("%s", s_basic);
    printk("Basic %s: [%s]\n", s_basic);

    halt();
}

```

其应当输出：

```
=== Basic Tests ===
+8906X
Basic %u: 8906
Finished 1st scan
K
Basic %c: 'K'
Finished 2nd scan
MOS_Kernel

Basic %s: [MOS_Kernel]
Finished 3rd scan
```

你可以使用：

- `make test lab=1_scan && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=1_scan && make run` 在本地测试上述样例（开启优化）

调试提示

由于本题目存在与控制台交互（输入）的需求，使用 `make dbg` 目标进行 GDB 调试时，无法直接向 QEMU 控制台完成输入。在这里提示两种使用 GDB 调试本题目实现的方法：

1. 使用 `make dbg_pts` 与 `make connect` 目标进行调试

在完成测试点编译后打开两个命令行窗口（可使用 `tmux`），先在其中一个执行 `make dbg_pts` 以启动 QEMU 并打开 GDB 调试界面，再在另一个终端中执行 `make connect` 连接到 QEMU 控制台进行测试输入。

2. 重定向 QEMU 终端输入

同学们也可以选择编辑 `tools/run_bg.sh` 脚本，将脚本第五行改为 `$QEMU $QEMU_FLAGS -kernel $mos_elf <./test.txt -s -S >/tmp/qemu_stdout &` 即将原先的 `/dev/null` 替换为重定向的输入文件 `./test.txt`。同学们只需在仓库根目录创建 `test.txt` 文件，并将测试输入写入其中即可（请注意，在测试输入末尾需要添加空白符标识输入结束）。注意，使用这种方式进行调试时，只可使用 `make dbg` 目标进行调试。

提交评测 & 评测标准

请在开发机中执行下列命令后，在课程网站上提交评测。

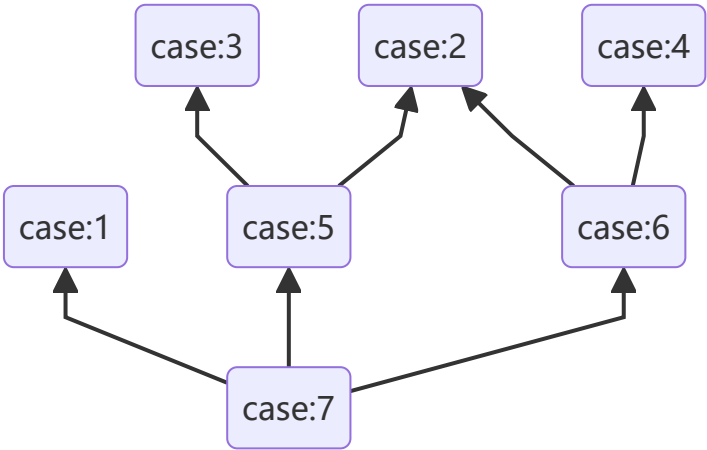
```
$ cd ~/学号/$ git add -A
$ git commit -m "complete lab1-extra scan"
$ git push
```

具体要求和分数分布如下：

测试点序号	评测内容	分值
1	与题面中的样例相同	10分

测试点序号	评测内容	分值
2	包含对 <code>%c</code> 基础格式的非混合评测	5 分
3	包含对 <code>%u</code> 基础格式的非混合评测（保证读取的数字之间有空格间隔）	10 分
4	包含对 <code>%s</code> 基础格式的非混合评测（保证读取的字符串之间有空格间隔）	10 分
5	包含对 <code>%c</code> 和 <code>%u</code> 相连格式的混合评测（不保证读取的字符串之间有空格间隔）	15 分
6	包含对 <code>%c</code> 和 <code>%s</code> 相连格式的混合评测（不保证读取的字符串之间有空格间隔）	20 分
7	综合组合评测	30 分

测试点依赖关系如下（箭头方向表示依赖方向）：



测试用例补充说明：

- 对于第一个测试点，其测试内容不包含样例外的任何其他内容。
- 对于后四个测试点，测试数据可能会包含**任何**题面中要求实现的功能变体及混合组合。
- 在线评测时，所有的 `.mk` 文件、所有的 `Makefile` 文件、`init/init.c` 以及 `tests/` 和 `tools/` 目录下的所有文件都可能被替换为标准版本，因此请同学们本地开发时，**不要**在这些文件中编写实际功能所依赖的代码。